Terraform challenge-2

Goal of the Exercise

The goal of this exercise is to test your understanding of a Cloud Compute deployment with Terraform.

You are tasked with creating a codebase for a microservice deployment on a cloud environment, using **Terraform**. Since our focus is not the application itself, the service will be a simple web server that returns a static page.

Requirements

Service Details

- The service will be a simple web server that returns a static page.
- You can use **apache2** or **nginx** to serve a simple HTML file.
- The service should be **containerized** and stored in a **Docker registry**.
- The service should be deployed on a **single VM instance** as a docker container.
- The image should be pulled from the registry specified earlier.
- The service should **not** be accessible from the internet but only through a **Load Balancer**, except for SSH access from specific IP addresses.

Deliverables

You are to provide the following as part of your solution:

1. **Dockerfile** to build the image.
2. **Terraform codebase** to deploy the following:
    - Docker registry
    - VPC and all required networking resources
    - EC2 instance
    - Load Balancer

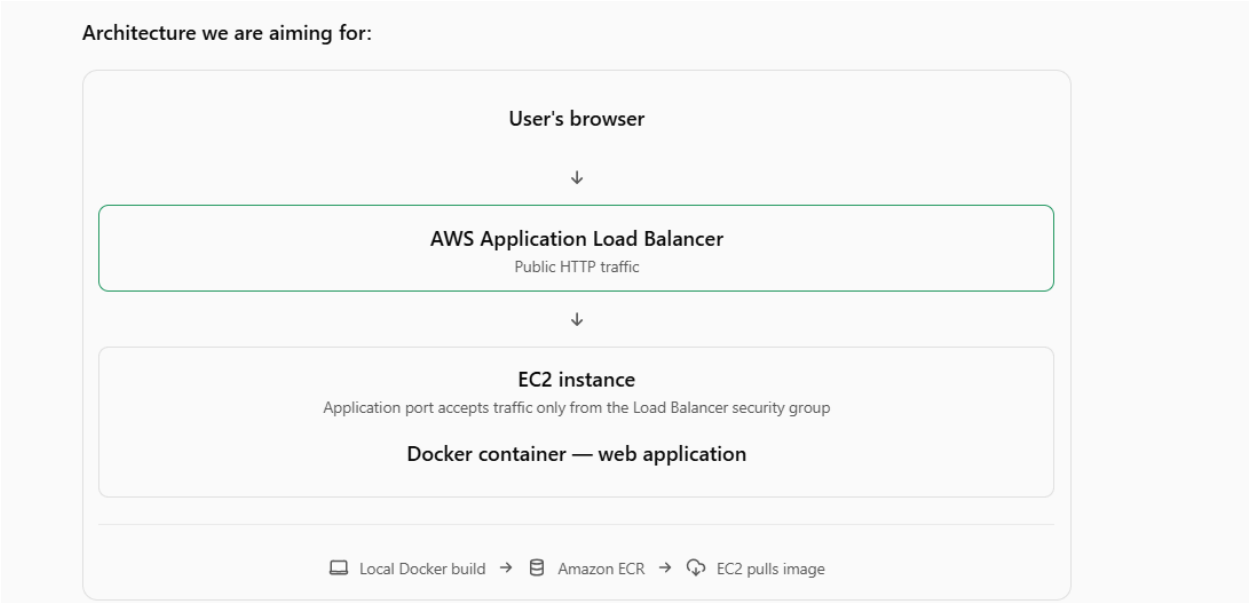
Notes

- Since a CI/CD pipeline is not part of this exercise, we will work around that with the help of Terraform.
- To push the image to the Docker registry, **build the image locally** before running the Terraform code.
- Once the image registry is created, use Terraform to **push the local image** to the registry.
- Similarly, use Terraform to **pull the image** from the registry and run it on the VM instance once it is created.
- For SSH access:
    - Use a locally created key.
    - Add it to the cloud provider key storage.
    - Attach it to the instance.
- Use **Terraform best practices** for file structure and code organization.
    - Use variables, locals, modules, etc. when appropriate.

- Include all `tfvars`, scripts, and other helper files used to deploy the application.
- You may use any cloud provider of your choosing, although **AWS is preferred**.
- Use a **free tier account** or a **trial account** to deploy the resources.

Project plan

1. Verify Terraform, AWS CLI, Docker, and AWS credentials.
2. Create a simple web page and Dockerfile.
3. Build and test the Docker image locally.
4. Create an ECR repository and push the image.
5. Use Terraform to create the VPC, networking, security groups, EC2, and Load Balancer.
6. Configure EC2 to pull the image from ECR and run the container.
7. Test access through the Load Balancer, document everything, and destroy the resources.



For this assignment, I'll use a public Load Balancer and a public-subnet EC2 instance with strict security-group rules: application traffic only from the Load Balancer, and SSH only from your allowed IP. This avoids the extra cost and complexity of a NAT Gateway while keeping the application inaccessible directly from the internet.

Step 1 — Check your local environment

Open Windows PowerShell and run these commands one by one:

```
None
terraform -version
aws --version
docker version
```

Now check which AWS account and region your CLI is using:

```
None
aws sts get-caller-identity
aws configure list
```

Finally, check that Docker Engine is running:

```
None
docker info
```

Step 2 — Create the project structure

We will use Nginx to serve a simple static web page inside a Docker container.

Open PowerShell and run:

```
None
$Project = "$HOME\terraform-microservice"

New-Item -ItemType Directory -Force -Path `
    "$Project\app", `
    "$Project\terraform" | Out-Null

Set-Location $Project
```

Our initial folder structure will be:

```
None
terraform-microservice/
│
├── app/
│   ├── index.html
│   └── Dockerfile
│
└── terraform/
```

2.1 Create the web page

Run:

```
None
@'
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Terraform Microservice</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f2f5f9;
            text-align: center;
            padding: 60px 20px;
        }
        .container {
            background: white;
            padding: 35px;
            border-radius: 12px;
            max-width: 650px;
            margin: auto;
            box-shadow: 0 4px 15px rgba(0,0,0,0.1);
        }
```

```
        h1 { color: #1769aa; }
        p { color: #444; line-height: 1.6; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Hello from AWS Microservice!</h1>
        <p>This web application is running inside a Docker
container.</p>
        <p>It is deployed on an EC2 instance using Terraform.</p>
        <p>Users access this application through an AWS Application
Load Balancer.</p>
    </div>
</body>
</html>
'@ | Set-Content -Path "$Project\app\index.html" -Encoding utf8
```

2.2 Create the Dockerfile

Run:

```
None
@'
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
'@ | Set-Content -Path "$Project\app\Dockerfile" -Encoding ascii
```

What does this Dockerfile do?

Instruction	Purpose

`FROM nginx:alpine`	Uses a lightweight Nginx base image
`COPY`	Copies our HTML page into Nginx's web directory
`EXPOSE 80`	Documents the port used by Nginx inside the container

2.3 Verify the files

Run:

```
None
Get-ChildItem -Recurse $Project
```

Then:

```
None
Get-Content "$Project\app\Dockerfile"
Get-Content "$Project\app\index.html"
```

Step 3 — Create the Dockerfile

Run this in your current PowerShell window:

```
None
@(
    "FROM nginx:alpine"
    ""
    "COPY index.html /usr/share/nginx/html/index.html"
    ""
```

```
    "EXPOSE 80"
) | Set-Content -Path "$Project\app\Dockerfile" -Encoding ascii
```

Understand each instruction

Instruction	Meaning
`FROM nginx:alpine`	Downloads a lightweight Nginx base image.
`COPY index.html ...`	Copies our web page into Nginx's default web directory.
`EXPOSE 80`	Documents that the container serves HTTP on port 80.

Why Nginx? It will serve our static HTML page when a user accesses the application.

Verify the project files

Run:

```
None
Get-ChildItem "$Project\app"
```

Then:

```
None
Get-Content "$Project\app\Dockerfile"
```

Expected Dockerfile:

```
None
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

Step 4 — Build the Docker image

First, move into the `app` directory:

```
None
Set-Location "$Project\app"
```

Build the image:

```
None
docker build -t terraform-microservice:v1 .
```

What happens here?

- `docker build` — builds an image using the Dockerfile.
- `-t terraform-microservice:v1` — assigns the image name and version.
- `.` — tells Docker to use the current directory as the build context. It contains both `Dockerfile` and `index.html`.

The first build downloads the `nginx:alpine` base image if it isn't already available locally.

Step 5 — Run the container

After the build succeeds, run:

```
None
docker run -d --name terraform-microservice-test -p 8088:80
terraform-microservice:v1
```

Understand the port mapping:

```
None
Your Windows machine                  Docker container
localhost:8088  ------------------->  Nginx:80
```

- `8088` is the port on your Windows machine.
- `80` is the port inside the container.
- `-d` runs the container in the background.

Step 6 — Verify the application

Check that the container is running:

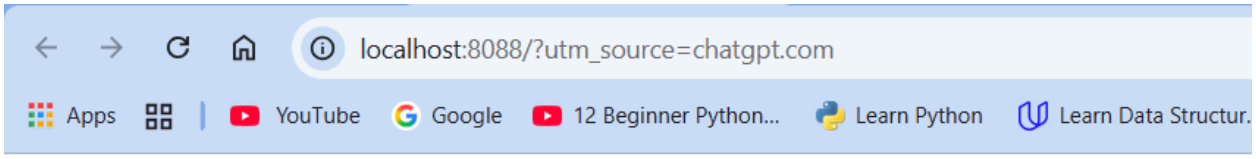
```
None
docker ps
```

Test the web page:

```
None
$response = Invoke-WebRequest http://localhost:8088
$response.StatusCode
$response.Content
```

Expected status:

```
None
200
```



Hello from AWS Microservice!

This application is running inside a Docker container.

Deployed on an EC2 instance using Terraform.

Accessed through an AWS Application Load Balancer.

Step 7 — Create the Amazon ECR repository with Terraform

Now we'll move from the local Docker test to AWS.

Amazon ECR (Elastic Container Registry) is a private Docker image registry. We'll push our image there, and later the EC2 instance will pull it from ECR.

Our first AWS resource will be only the ECR repository. We will create the network, EC2, and Load Balancer in later steps.

We'll use the AWS Terraform provider 6.x and its `aws_ecr_repository` resource.

Terraform Registry+1

7.1 Create the Terraform files

In PowerShell, run:

```
None
$Project = "C:\Users\HP\terraform-microservice"
$TerraformDir = Join-Path $Project "terraform"

Set-Location $TerraformDir
```

Create these files in the `terraform` folder:

```
None
terraform/
├── versions.tf
├── provider.tf
├── variables.tf
├── locals.tf
├── ecr.tf
├── outputs.tf
└── terraform.tfvars
```

A. `versions.tf` — Terraform and provider requirements

```
None
@'
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
'@ | Set-Content -Path ".\versions.tf" -Encoding ascii
```

B. `provider.tf` — Connect Terraform to AWS

```
None
@'
provider "aws" {
  region  = var.aws_region
  profile = var.aws_profile

  default_tags {
```

```
    tags = local.common_tags
  }
}
'@ | Set-Content -Path ".\provider.tf" -Encoding ascii
```

This uses your existing AWS CLI profile, `terraform-lab`. We are not putting access keys or secret keys in Terraform files.

C. `variables.tf` — Inputs for the project

```
None
@'
variable "aws_region" {
  description = "AWS region for this project"
  type        = string
}

variable "aws_profile" {
  description = "AWS CLI profile used by Terraform"
  type        = string
}

variable "project_name" {
  description = "Name used for project resources"
  type        = string
}
'@ | Set-Content -Path ".\variables.tf" -Encoding ascii
```

D. `locals.tf` — Common resource tags

```
None
@'
locals {
  common_tags = {
    Project     = var.project_name
    Environment = "training"
```

```
    ManagedBy   = "Terraform"
  }
}
'@ | Set-Content -Path ".\locals.tf" -Encoding ascii
```

E. `ecr.tf` — Create the ECR repository

```
None
@'
resource "aws_ecr_repository" "app" {
  name                 = "${var.project_name}-app"
  image_tag_mutability = "MUTABLE"
  force_delete         = true

  image_scanning_configuration {
    scan_on_push = true
  }
}
'@ | Set-Content -Path ".\ecr.tf" -Encoding ascii
```

A few important settings:

- `MUTABLE` allows us to push an updated image using the same tag during practice.
- `scan_on_push = true` requests image scanning when an image is pushed.
- `force_delete = true` allows Terraform to delete this training repository even when it contains images. Be careful: destroying it will delete those images too.

F. `outputs.tf` — Display the repository details after creation

```
None
@'
output "ecr_repository_name" {
  description = "Name of the ECR repository"
  value       = aws_ecr_repository.app.name
}
```

```
output "ecr_repository_url" {
  description = "ECR URL used to push and pull Docker images"
  value       = aws_ecr_repository.app.repository_url
}

output "ecr_repository_arn" {
  description = "ARN of the ECR repository"
  value       = aws_ecr_repository.app.arn
}
'@ | Set-Content -Path ".\outputs.tf" -Encoding ascii
```

G. `terraform.tfvars` — Supply the project values

```
None
@'
aws_region   = "us-east-1"
aws_profile  = "terraform-lab"
project_name = "terraform-microservice"
'@ | Set-Content -Path ".\terraform.tfvars" -Encoding ascii
```

7.2 Verify the files

Run:

```
None
Get-ChildItem
```

You should see all seven Terraform files.

7.3 Initialize and validate Terraform

Run these commands one at a time, from the `terraform` directory:

```
None
terraform fmt -recursive
```

```
None
terraform init
```

```
None
terraform validate
```

Then create a plan:

```
None
terraform plan
```

The plan should show 1 resource to add: `aws_ecr_repository.app`.

Step 8 — Create the ECR repository

Run:

```
None
terraform apply
```

Terraform will display the plan again. Review it and type:

```
None
yes
```

Terraform should finish with:

```
None
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Step 9 — Verify the repository

After the apply succeeds, run:

```
None
terraform output
```

Then verify directly through AWS CLI:

```
None
aws ecr describe-repositories `
  --repository-names terraform-microservice-app `
  --region us-east-1
```

Step 10 — Push the Docker image to ECR using Terraform

Currently, the image exists locally on your Windows machine:

```
None
terraform-microservice:v1
```

We need to push it to ECR. We'll use Terraform's built-in `terraform_data` resource with a `local-exec` provisioner. This runs AWS CLI and Docker commands on your Windows machine when Terraform applies the configuration.

The workflow:



10.1 Verify the local image

Run:

```
None
docker image inspect terraform-microservice:v1 --format '{{.Id}}'
```

It should return the image ID.

10.2 Add two variables

Make sure your current directory is:

```
None
C:\Users\HP\terraform-microservice\terraform
```

Append these variables to `variables.tf`:

```
None
@'

variable "image_name" {
  description = "Local Docker image name"
  type        = string
  default     = "terraform-microservice"
}

variable "image_tag" {
  description = "Docker image tag"
  type        = string
  default     = "v1"
}
'@ | Add-Content -Path ".\variables.tf" -Encoding ascii
```

10.3 Create `image_push.tf`

Run this complete command:

None

```
@'
resource "terraform_data" "push_image" {
  triggers_replace = {
    repository_url = aws_ecr_repository.app.repository_url
    image_name     = var.image_name
    image_tag      = var.image_tag
  }

  provisioner "local-exec" {
    interpreter = [
      "powershell.exe",
      "-NoProfile",
      "-ExecutionPolicy",
      "Bypass",
      "-Command"
    ]

    command = <<-EOT
      $ErrorActionPreference = "Stop"

      $env:AWS_PROFILE = "${var.aws_profile}"
      $env:AWS_DEFAULT_PROFILE = "${var.aws_profile}"

      $registry =
"${aws_ecr_repository.app.repository_url}".Split("/")[0]

      $password = aws ecr get-login-password --region
"${var.aws_region}"
      if ($LASTEXITCODE -ne 0) { throw "Could not obtain ECR login
token." }

      $password | docker login --username AWS --password-stdin
$registry
      if ($LASTEXITCODE -ne 0) { throw "Docker login failed." }
```

```
      docker tag "${var.image_name}:${var.image_tag}"
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"
      if ($LASTEXITCODE -ne 0) { throw "Docker tagging failed." }

      docker push
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"
      if ($LASTEXITCODE -ne 0) { throw "Docker push failed." }
    EOT
  }
}
'@ | Set-Content -Path ".\image_push.tf" -Encoding ascii
```

Understand what Terraform will execute:

Command	Purpose
`aws ecr get-login-password`	Gets a temporary ECR authentication token
`docker login`	Authenticates Docker to your ECR registry
`docker tag`	Adds the ECR repository URI to your local image
`docker push`	Uploads the image to ECR

The provisioner uses your existing `terraform-lab` AWS profile. It does not place AWS access keys or the temporary token in the Terraform configuration.

10.4 Format, validate, and plan

Run:

```
None
terraform fmt
terraform validate
terraform plan
```

Step 11 — Push the image to ECR

Run:

```
None
terraform apply
```

Review the plan and type `yes` when prompted.

Terraform should execute these operations in order:

1. Get an ECR authentication token.
2. Log Docker into Amazon ECR.
3. Tag your local image with the ECR repository URL.
4. Push the image to ECR.

Expected final message:

```
None
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Step 12 — Verify the image in ECR

After the apply succeeds, run:

```
None
aws ecr list-images `
  --repository-name terraform-microservice-app `
  --region us-east-1
```

Expected result should include:

```
None
{
    "imageIds": [
        {
            "imageTag": "v1"
        }
    ]
}
```

You can also check the image locally:

```
None
docker images
```

Step 11A — Test the PowerShell encoding fix

Run these commands in your current PowerShell window.

1. Check your PowerShell version:

```
None
$PSVersionTable.PSVersion
```

2. Set console input encoding to UTF-8 without BOM:

```
None
[Console]::InputEncoding = New-Object System.Text.UTF8Encoding
$false
```

3. Try ECR login again:

```
None
aws ecr get-login-password --region us-east-1 --profile
terraform-lab |
docker login --username AWS --password-stdin
986881030460.dkr.ecr.us-east-1.amazonaws.com
```

Expected output

```
None
Login Succeeded
```

This encoding change applies to the current PowerShell session.

We have identified the issue: the ECR authentication token was not being passed correctly from PowerShell to Docker. Your encoding adjustment fixed it.

One important point: Terraform runs its own separate PowerShell process, so we must include that encoding adjustment inside `image_push.tf` too.

Step 12 — Fix `image_push.tf`

Replace the existing file with this corrected version:

```
None
@'
resource "terraform_data" "push_image" {
  triggers_replace = {
    repository_url = aws_ecr_repository.app.repository_url
    image_name     = var.image_name
    image_tag      = var.image_tag
  }

  provisioner "local-exec" {
    interpreter = [
      "powershell.exe",
      "-NoProfile",
      "-ExecutionPolicy",
      "Bypass",
      "-Command"
    ]

    command = <<-EOT
      [Console]::InputEncoding = New-Object
System.Text.UTF8Encoding $false
      $ErrorActionPreference = "Stop"
```

```
      $env:AWS_PROFILE = "${var.aws_profile}"
      $env:AWS_DEFAULT_PROFILE = "${var.aws_profile}"

      $registry =
"${aws_ecr_repository.app.repository_url}".Split("/")[0]

      aws ecr get-login-password --region "${var.aws_region}" |
        docker login --username AWS --password-stdin $registry
      if ($LASTEXITCODE -ne 0) { throw "ECR Docker login failed." }

      docker tag "${var.image_name}:${var.image_tag}"
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"
      if ($LASTEXITCODE -ne 0) { throw "Docker tagging failed." }

      docker push
"${aws_ecr_repository.app.repository_url}:${var.image_tag}"
      if ($LASTEXITCODE -ne 0) { throw "Docker push failed." }
    EOT
  }
}
'@ | Set-Content -Path ".\image_push.tf" -Encoding ascii
```

Step 13 — Validate the updated configuration

Run:

```
None
terraform fmt
terraform validate
terraform plan
```

Terraform has marked `terraform_data.push_image` as tainted, so it must replace that helper resource and retry its commands.

Understand the plan

Action	Resource
Destroy and replace	`terraform_data.push_image`
Create	New `terraform_data.push_image`
Modify	0
ECR repository changes	None

Important: The destroy action applies only to the Terraform helper resource. Your ECR repository will not be deleted.

Step 14 — Retry the image push

Run:

```
terraform apply
```

Review the plan and enter:

```
yes
```

Look for these messages:

```
None
Login Succeeded
```

```
None
The push refers to repository [...]
```

And finally:

```
None
Apply complete!
```

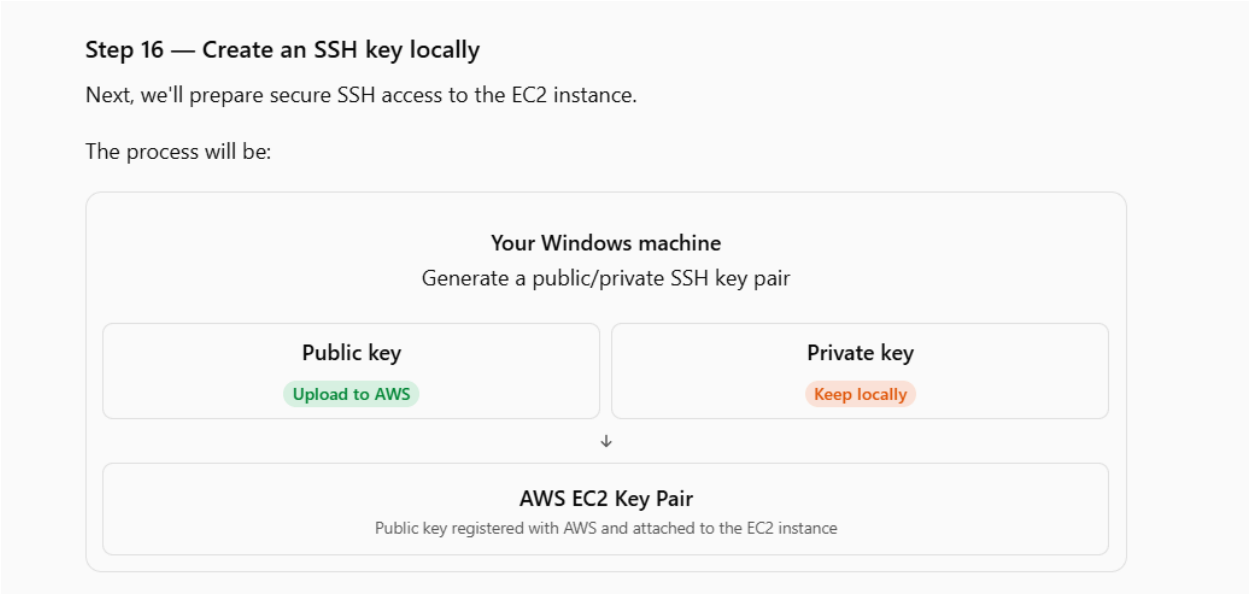
Step 15 — Verify the image

After a successful apply, run:

```
None
aws ecr list-images `
  --repository-name terraform-microservice-app `
  --region us-east-1
```

Expected result:

```
None
{
    "imageIds": [
        {
            "imageTag": "v1"
        }
    ]
}
```



16.1 Check SSH key-generation utility

Run:

```
None
Get-Command ssh-keygen
```

16.2 Create the keys folder

```
None
$Project = "C:\Users\HP\terraform-microservice"

New-Item -ItemType Directory -Force -Path "$Project\keys" |
Out-Null
```

16.3 Generate an Ed25519 key pair

```
None
$keyPath = "$Project\keys\terraform-microservice-key"

if (-not (Test-Path $keyPath)) {
    ssh-keygen -t ed25519 -C "terraform-microservice" -f $keyPath
} else {
```

```
    Write-Host "Key already exists. Not overwriting."
}
```

At the passphrase prompt, you can set a passphrase to protect your private key. Do not share it.

16.4 Verify the files

```
None
Get-ChildItem "$Project\keys"
```

Expected:

```
None
terraform-microservice-key
terraform-microservice-key.pub
```

The file without `.pub` is your private key. Never upload or share it. We'll register only the `.pub` file with AWS.

Step 17 — Find your public IP for SSH access

We will allow SSH only from your current public IP using a `/32` CIDR.

Run:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()

Write-Host "SSH allowed CIDR: $MyPublicIp/32"
```

Both SSH key files were generated successfully, and your public IP check also worked.

SSH preparation complete

Item	Result

SSH key type	Ed25519
Private key	`keys\terraform-microservice-key`
Public key	`keys\terraform-microservice-key.pub`
SSH allowed CIDR	`124.123.159.234/32`
AWS region	`us-east-1`

Important: Never upload or share the private key. Terraform will register only the `.pub` file with AWS. Your SSH access is restricted to the IP above. When your public IP changes, we'll update the CIDR.

Step 18 — Create our custom AWS network

We'll create a dedicated VPC rather than using the default VPC.

Our network design:

VPC — 10.20.0.0/16

Internet Gateway

Public route table

`0.0.0.0/0 → Internet Gateway`

Public subnet A

10.20.1.0/24

Availability Zone 1

Public subnet B

10.20.2.0/24

Availability Zone 2

Both subnets will be associated with the public route table. The two subnets will later host our Application Load Balancer.

Why two subnets? An internet-facing Application Load Balancer requires subnets in at least two Availability Zones.

18.1 Add network variables

You are already in the `terraform` directory. Run:

```
None
@'

variable "vpc_cidr" {
  description = "CIDR block for the project VPC"
  type        = string
  default     = "10.20.0.0/16"
}

variable "public_subnet_a_cidr" {
  description = "CIDR block for public subnet A"
  type        = string
  default     = "10.20.1.0/24"
}

variable "public_subnet_b_cidr" {
  description = "CIDR block for public subnet B"
  type        = string
  default     = "10.20.2.0/24"
}

'@ | Add-Content -Path ".\variables.tf" -Encoding ascii
```

18.2 Create `network.tf`

Run this complete block:

```
None
@'
data "aws_availability_zones" "available" {
  state = "available"
}

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.project_name}-vpc"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "${var.project_name}-igw"
  }
}

resource "aws_subnet" "public_a" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_a_cidr
  availability_zone       =
data.aws_availability_zones.available.names[0]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-a"
  }
}
```

```
resource "aws_subnet" "public_b" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_b_cidr
  availability_zone       =
data.aws_availability_zones.available.names[1]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-b"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "${var.project_name}-public-rt"
  }
}

resource "aws_route_table_association" "public_a" {
  subnet_id      = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "public_b" {
  subnet_id      = aws_subnet.public_b.id
  route_table_id = aws_route_table.public.id
}
'@ | Set-Content -Path ".\network.tf" -Encoding ascii
```

18.3 Create network outputs

Append the following to `outputs.tf`:

```
None
@'

output "vpc_id" {
  description = "Project VPC ID"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs of the two public subnets"
  value = [
    aws_subnet.public_a.id,
    aws_subnet.public_b.id
  ]
}

output "availability_zones" {
  description = "Availability Zones used by the public subnets"
  value = [
    aws_subnet.public_a.availability_zone,
    aws_subnet.public_b.availability_zone
  ]
}

output "internet_gateway_id" {
  value = aws_internet_gateway.main.id
}
'@ | Add-Content -Path ".\outputs.tf" -Encoding ascii
```

18.4 Format and validate

Run these commands individually:

```
None
terraform fmt -recursive
```

Step 19 — Create the AWS network

Run:

```
None
terraform apply
```

Review the plan and enter:

```
None
yes
```

Expected result:

```
None
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

Step 20 — Verify the network

After successful creation, run:

```
None
terraform output
```

Then:

```
None
aws ec2 describe-vpcs `
  --filters "Name=tag:Name,Values=terraform-microservice-vpc" `
  --region us-east-1
```

And:

```
None
aws ec2 describe-subnets `
```

```
  --filters "Name=vpc-id,Values=$(terraform output -raw vpc_id)" `
  --region us-east-1 `
  --query
"Subnets[*].[SubnetId,CidrBlock,AvailabilityZone,MapPublicIpOnLaunc
h]" `
  --output table
```

We should see two subnets in different Availability Zones.

Step 21 — Verify the subnets and public route

You have already verified the VPC. Let's verify the remaining network details.

Run:

```
None
$VpcId = terraform output -raw vpc_id

aws ec2 describe-subnets `
  --filters "Name=vpc-id,Values=$VpcId" `
  --region us-east-1 `
  --query
"Subnets[*].[SubnetId,CidrBlock,AvailabilityZone,MapPublicIpOnLaunc
h]" `
  --output table
```

Expected subnet information:

Subnet	CIDR	Availability Zone
Public A	`10.20.1.0/24`	`us-east-1a`

Public B	`10.20.2.0/24`	`us-east-1b`

Next, verify the Internet Gateway attachment:

```
None
aws ec2 describe-internet-gateways `
  --filters "Name=attachment.vpc-id,Values=$VpcId" `
  --region us-east-1 `
  --query
"InternetGateways[*].[InternetGatewayId,Attachments[0].State]" `
  --output table
```

Expected state: `available`.

Finally, verify the default route:

```
None
aws ec2 describe-route-tables `
  --filters "Name=vpc-id,Values=$VpcId" `
  --region us-east-1 `
  --query "RouteTables[*].Routes" `
  --output json
```

Look for:

```
None
DestinationCidrBlock: 0.0.0.0/0
GatewayId: igw-040f35a9618fe9d62
State: active
```

```
None
terraform validate
```

```
None
terraform plan
```

All three checks are successful:

- Subnets: Two public subnets, in `us-east-1a` and `us-east-1b`.
- Internet Gateway: Attached and available.
- Routing: The custom public route table has an active `0.0.0.0/0` route to the Internet Gateway.

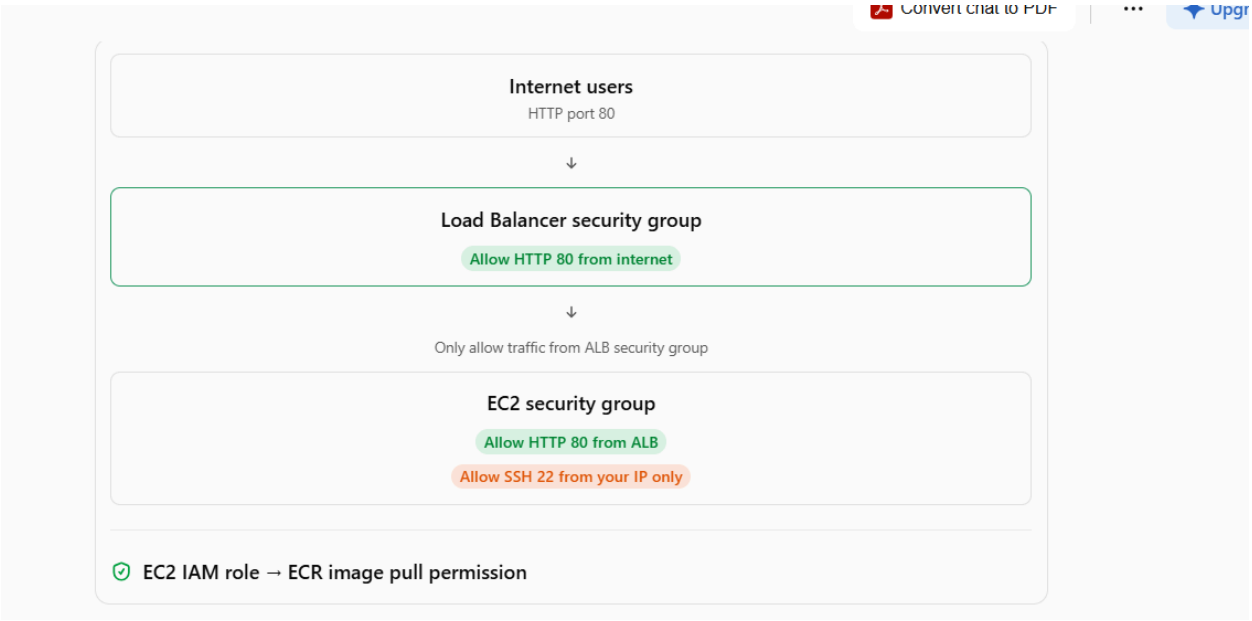
The route-table output also shows the VPC's main route table with only its local route. That's normal; your two public subnets are associated with the separate route table that has the Internet Gateway route.

Step 22 — Create security groups, IAM role and EC2 key pair

Before launching EC2, we need to prepare three things:

1. Security groups: Control which traffic can reach the Load Balancer and EC2.
2. IAM instance profile: Gives EC2 permission to pull the Docker image from ECR without storing AWS access keys on the server.
3. EC2 key pair: Registers your public SSH key with AWS.

Our security design:



We'll use separate Terraform security-group rule resources rather than mixing inline and standalone rules.

Terraform Registry+1
AWS will register only the public key material from your SSH key pair.
Terraform Registry

22.1 Add variables

Run this in your current PowerShell window:

```
None
@'

variable "ssh_allowed_cidr" {
  description = "Public IP CIDR allowed to SSH into EC2"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
'@ | Add-Content -Path ".\variables.tf" -Encoding ascii
```

Now refresh your public IP and add it to `terraform.tfvars`:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()

Add-Content -Path ".\terraform.tfvars" `
  -Value "`nssh_allowed_cidr = `"$MyPublicIp/32`""

Write-Host "SSH CIDR configured: $MyPublicIp/32"
```

This keeps SSH access restricted to your current public IP. If that IP changes, update the value before applying future changes.

22.2 Create `security.tf`

Run:

```
None
@'
# Security group for the public Application Load Balancer
resource "aws_security_group" "alb" {
  name        = "${var.project_name}-alb-sg"
  description = "Allow public HTTP traffic to the Application Load
Balancer"
  vpc_id      = aws_vpc.main.id

  egress = []

  tags = {
    Name = "${var.project_name}-alb-sg"
  }
}

# Security group for the EC2 application server
resource "aws_security_group" "ec2" {
  name        = "${var.project_name}-ec2-sg"
  description = "Allow application traffic only from ALB and SSH
from approved IP"
  vpc_id      = aws_vpc.main.id

  egress = []

  tags = {
    Name = "${var.project_name}-ec2-sg"
  }
}

# Public users can access ALB on HTTP port 80
resource "aws_vpc_security_group_ingress_rule" "alb_http" {
  security_group_id = aws_security_group.alb.id
  cidr_ipv4         = "0.0.0.0/0"
```

```
  from_port         = 80
  to_port           = 80
  ip_protocol       = "tcp"
  description       = "Allow public HTTP traffic"
}

# ALB can send HTTP traffic to EC2
resource "aws_vpc_security_group_egress_rule" "alb_to_ec2" {
  security_group_id            = aws_security_group.alb.id
  referenced_security_group_id = aws_security_group.ec2.id
  from_port                    = 80
  to_port                      = 80
  ip_protocol                  = "tcp"
  description                  = "Forward HTTP traffic to EC2"
}

# EC2 accepts HTTP only from the ALB security group
resource "aws_vpc_security_group_ingress_rule" "ec2_http_from_alb"
{
  security_group_id            = aws_security_group.ec2.id
  referenced_security_group_id = aws_security_group.alb.id
  from_port                    = 80
  to_port                      = 80
  ip_protocol                  = "tcp"
  description                  = "Allow application HTTP traffic
only from ALB"
}

# SSH access only from your approved public IP
resource "aws_vpc_security_group_ingress_rule" "ec2_ssh" {
  security_group_id = aws_security_group.ec2.id
  cidr_ipv4         = var.ssh_allowed_cidr
  from_port         = 22
  to_port           = 22
  ip_protocol       = "tcp"
  description       = "SSH access from approved administrator IP"
```

```
}

# EC2 requires outbound access for packages and ECR image downloads
resource "aws_vpc_security_group_egress_rule" "ec2_outbound" {
  security_group_id = aws_security_group.ec2.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1"
  description       = "Allow outbound access for bootstrap and ECR
image pull"
}
'@ | Set-Content -Path ".\security.tf" -Encoding ascii
```

Why these rules matter:

Traffic	Allowed source/destination
Internet → ALB	HTTP 80
ALB → EC2	HTTP 80
Your IP → EC2	SSH 22
Other internet users → EC2 application	Blocked

22.3 Create `iam.tf`

EC2 needs permission to download the private Docker image from ECR.

Instead of storing AWS access keys on EC2, we'll attach an IAM role through an instance profile.

Run:

```
None
@'
resource "aws_iam_role" "ec2_ecr_pull" {
  name = "${var.project_name}-ec2-ecr-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Effect = "Allow"

        Principal = {
          Service = "ec2.amazonaws.com"
        }

        Action = "sts:AssumeRole"
      }
    ]
  })

  tags = local.common_tags
}

resource "aws_iam_role_policy" "ec2_ecr_pull" {
  name = "${var.project_name}-ecr-pull-policy"
  role = aws_iam_role.ec2_ecr_pull.id

  policy = jsonencode({
    Version = "2012-10-17"

    Statement = [
      {
        Sid      = "ECRAuthorization"
        Effect   = "Allow"
        Action   = ["ecr:GetAuthorizationToken"]
        Resource = "*"
```

```
      },
      {
        Sid    = "PullApplicationImage"
        Effect = "Allow"

        Action = [
          "ecr:BatchCheckLayerAvailability",
          "ecr:BatchGetImage",
          "ecr:GetDownloadUrlForLayer"
        ]

        Resource = aws_ecr_repository.app.arn
      }
    ]
  })
}

resource "aws_iam_instance_profile" "ec2" {
  name = "${var.project_name}-ec2-profile"
  role = aws_iam_role.ec2_ecr_pull.name

  tags = local.common_tags
}
'@ | Set-Content -Path ".\iam.tf" -Encoding ascii
```

Remember this relationship:

```
None
EC2 instance
     |
     v
IAM Instance Profile
     |
     v
IAM Role
     |
```

```
     v
ECR Pull Permissions
     |
     v
Amazon ECR
     |
     v
Docker image downloaded
```

22.4 Create `keypair.tf`

This registers your existing public key with AWS.

```
None
@'
resource "aws_key_pair" "app" {
  key_name   = "${var.project_name}-key"
  public_key =
file("${path.module}/../keys/terraform-microservice-key.pub")

  tags = {
    Name = "${var.project_name}-key"
  }
}
'@ | Set-Content -Path ".\keypair.tf" -Encoding ascii
```

The private key remains on your Windows machine. Only the `.pub` file is registered with AWS.

22.5 Add outputs

Append these to `outputs.tf`:

```
None
@'

output "alb_security_group_id" {
```

```
  value = aws_security_group.alb.id
}

output "ec2_security_group_id" {
  value = aws_security_group.ec2.id
}

output "ec2_instance_profile_name" {
  value = aws_iam_instance_profile.ec2.name
}

output "ec2_key_pair_name" {
  value = aws_key_pair.app.key_name
}
'@ | Add-Content -Path ".\outputs.tf" -Encoding ascii
```

Step 23 — Validate and review

Run:

None
```
terraform fmt -recursive
```

None
```
terraform validate
```

None
```
terraform plan
```

Step 23 — Plan review

Resource	Count

Security groups	2
Security-group rules	5
IAM role	1
IAM policy	1
IAM instance profile	1
EC2 key pair	1
Total	11

Terraform reports:

```
None
Plan: 11 to add, 0 to change, 0 to destroy.
```

Pasted text(8).txt

Security review

Your rules are correct:

- ALB accepts HTTP traffic from the internet on port 80.
- EC2 accepts application traffic only from the ALB security group on port 80.
- SSH is allowed only from `124.123.159.234/32`.
- EC2 gets outbound access to download its image from ECR.
- EC2 receives ECR permissions through an IAM role, not stored AWS credentials.
  Pasted text(8).txt

  Pasted text(8).txt

The public key shown in the plan is safe to register with AWS. Your private key remains on your computer.

Step 24 — Create the security and IAM resources

Before applying, make sure your public IP has not changed. Then run:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()
Write-Host "Current IP: $MyPublicIp"
```

It should match the IP configured in `terraform.tfvars`.

Now run:

```
None
terraform apply
```

Review the plan and type:

```
None
yes
```

Expected:

```
None
Apply complete! Resources: 11 added, 0 changed, 0 destroyed.
```

Important resource IDs from your output:

- VPC: `vpc-0cee5b330b7779857`
- ALB security group: `sg-019c05f7b2a7b369e`
- EC2 security group: `sg-0d2957b61d867ed92`
- EC2 instance profile: `terraform-microservice-ec2-profile`
- SSH key pair: `terraform-microservice-key`

Step 25 — Prepare the EC2 instance

We'll use Amazon Linux 2023, install Docker automatically through EC2 user data, pull the private image from ECR using the IAM role, and run the container on port 80.

Our intended flow:

```
None
EC2 starts
   |
   v
Install Docker
   |
   v
Get ECR authentication using IAM role
   |
   v
Pull terraform-microservice-app:v1
   |
   v
Run Docker container on port 80
```

The EC2 instance will have a public IP for restricted SSH access, but its application port 80 will accept traffic only from the ALB security group.

25.1 Verify the Amazon Linux AMI

Before creating the instance, check that AWS can find the latest Amazon Linux 2023 x86_64 AMI:

```
None
aws ssm get-parameter `
  --name
"/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86
_64" `
  --region us-east-1 `
  --query "Parameter.Value" `
  --output text
```

We'll use this Systems Manager public parameter rather than hardcoding an AMI ID. It resolves to the latest supported default Amazon Linux 2023 image in the selected region.

Amazon Elastic Compute Cloud+1

25.2 Create `user_data.sh`

This script runs automatically when EC2 starts.

It will:

1. Install Docker.
2. Install AWS CLI v2 if it isn't already available.
3. Authenticate to ECR using the EC2 IAM role.
4. Pull the `v1` image.
5. Start the container on port 80.

Amazon Linux 2023 uses DNF, and AWS documents Docker installation through its package manager.

AWS Serverless Application Model+1

Run:

```
None
@'
#!/bin/bash
set -euxo pipefail

exec > >(tee -a /var/log/user-data.log) 2>&1

echo "Starting microservice EC2 bootstrap"

# Install Docker and required utilities
dnf install -y docker curl unzip

# Start Docker automatically
systemctl enable --now docker

# Install AWS CLI v2 if not already installed
if ! command -v aws >/dev/null 2>&1; then
```

```
    curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash -s
-- --system
fi

aws --version
docker --version

# Allow ec2-user to use Docker after reconnecting
usermod -aG docker ec2-user

# Authenticate Docker to Amazon ECR using the EC2 IAM role
login_success=false

for attempt in {1..12}; do
    if aws ecr get-login-password --region "${aws_region}" |
        docker login --username AWS --password-stdin
"${ecr_registry}"; then
        login_success=true
        break
    fi

    echo "ECR login attempt $attempt failed. Retrying..."
    sleep 10
done

if [[ "$login_success" != "true" ]]; then
    echo "ECR authentication failed"
    exit 1
fi

# Download the application image
docker pull "${ecr_repository_url}:${image_tag}"

# Start the application container
docker run -d \
    --name terraform-microservice \
```

```
    --restart unless-stopped \
    -p 80:80 \
    "${ecr_repository_url}:${image_tag}"

# Verify the container
docker ps
curl -f http://127.0.0.1/

echo "Microservice deployment completed successfully"
'@ | Set-Content -Path ".\user_data.sh" -Encoding ascii
```

25.3 Create `ec2.tf`

This file defines the EC2 instance and connects it to all the resources we created earlier.

Run:

```
None
@'
data "aws_ssm_parameter" "al2023_ami" {
  name =
"/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86
_64"
}

resource "aws_instance" "app" {
  ami           = data.aws_ssm_parameter.al2023_ami.value
  instance_type = var.instance_type

  subnet_id = aws_subnet.public_a.id

  vpc_security_group_ids = [
    aws_security_group.ec2.id
  ]

  key_name              = aws_key_pair.app.key_name
```

```
  iam_instance_profile = aws_iam_instance_profile.ec2.name

  associate_public_ip_address = true

  user_data = templatefile("${path.module}/user_data.sh", {
    aws_region        = var.aws_region
    ecr_registry      = split("/",
aws_ecr_repository.app.repository_url)[0]
    ecr_repository_url = aws_ecr_repository.app.repository_url
    image_tag         = var.image_tag
  })

  user_data_replace_on_change = true

  metadata_options {
    http_endpoint = "enabled"
    http_tokens   = "required"
  }

  root_block_device {
    volume_size           = 8
    volume_type           = "gp3"
    encrypted             = true
    delete_on_termination = true
  }

  depends_on = [
    aws_iam_role_policy.ec2_ecr_pull,
    aws_route_table_association.public_a
  ]

  tags = {
    Name = "${var.project_name}-ec2"
  }
}
'@ | Set-Content -Path ".\ec2.tf" -Encoding ascii
```

Why the `depends_on`?

The EC2 instance must wait until the IAM policy is attached and its subnet has public routing. Otherwise, the bootstrap script might start before the resources it needs are ready.

25.4 Add EC2 outputs

Append these to `outputs.tf`:

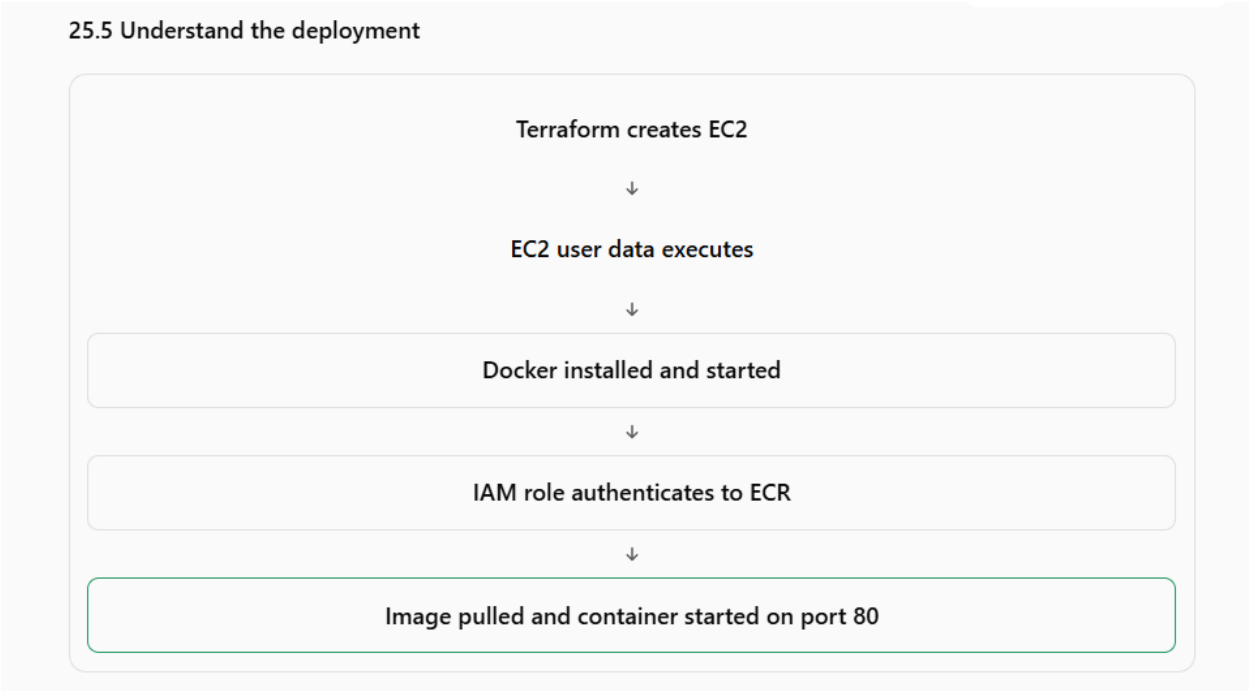
```
None
@'

output "ec2_instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.app.id
}

output "ec2_public_ip" {
  description = "EC2 public IP for restricted SSH access"
  value       = aws_instance.app.public_ip
}

output "ec2_public_dns" {
  description = "EC2 public DNS"
  value       = aws_instance.app.public_dns
}

output "ec2_ami_id" {
  description = "Amazon Linux 2023 AMI used"
  value       = aws_instance.app.ami
}
'@ | Add-Content -Path ".\outputs.tf" -Encoding ascii
```



25.6 Validate and plan

Run these commands individually:

None
```
terraform fmt -recursive
```

None
```
terraform validate
```

None
```
terraform plan
```

sue 1 — Security group egress conflicts

Your plan shows:

```
None
aws_security_group.alb will be updated in-place
aws_security_group.ec2 will be updated in-place

Plan: 1 to add, 2 to change, 0 to destroy.
```

The cause is that we used:

```
None
egress = []
```

inside `aws_security_group`, while also managing outbound rules with separate `aws_vpc_security_group_egress_rule` resources.

The AWS provider warns against mixing inline security group rules with standalone security group rule resources because it can cause conflicts and perpetual changes.

GitHub+1

Fix it

Run this PowerShell command from your `terraform` directory:

```
None
$content = Get-Content ".\security.tf" -Raw

$content = $content -replace '(?m)^[ \t]*egress[ \t]*=[ \t]*\[\][ \t]*\r?\n', ''

Set-Content ".\security.tf" -Value $content -Encoding ascii
```

This removes only the two inline `egress = []` declarations. It preserves our five standalone security group rules.

Issue 2 — Sensitive AMI output

The second error is:

```
None
Error: Output refers to sensitive values

on outputs.tf line 72:
output "ec2_ami_id"
```

Why?

The AWS provider marks `aws_ssm_parameter.value` as sensitive even when the parameter contains an ordinary public AMI ID. That sensitivity propagates into the EC2 AMI attribute.

Terraform Registry

An AMI ID is public information, so we can explicitly remove that sensitivity marking.

Run:

```
None
$content = Get-Content ".\outputs.tf" -Raw

$content = $content -replace 'value\s*=\s*aws_instance\.app\.ami',
'value = nonsensitive(aws_instance.app.ami)'

Set-Content ".\outputs.tf" -Value $content -Encoding ascii
```

The output block should now look like this:

```
None
output "ec2_ami_id" {
  description = "Amazon Linux 2023 AMI used"
  value       = nonsensitive(aws_instance.app.ami)
}
```

Step 3 — Validate the fixes

Run these commands one by one:

```
None
terraform fmt -recursive
```

```
None
terraform validate
```

Check that the old inline egress declarations are gone:

```
None
Select-String -Path ".\security.tf" -Pattern 'egress\s*=\s*\[\]'
```

No output is expected.

Finally:

```
None
terraform plan
```

Expected plan

```
None
Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform will create only `aws_instance.app`. Your existing VPC, subnets, security groups, IAM role, SSH key pair, and ECR repository will remain unchanged.

The selected configuration is:

- Instance type: `t3.micro`
- AMI: Amazon Linux 2023, `ami-0c230065b91270bc6`
- Subnet: `us-east-1a`
- Root disk: 8 GB encrypted `gp3`
- SSH: restricted to your configured `/32` IP
- Application port 80: accepts traffic only from the ALB security group

I also checked the EC2 bootstrap commands. Amazon Linux 2023 includes AWS CLI v2, and AWS documents installing Docker with `dnf install docker` and starting it with `systemctl enable --now docker`. Your script follows that approach.

Amazon Linux 2023+1

Step 26 — Launch EC2

Before applying, verify that your public IP has not changed:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()
Write-Host "Current public IP: $MyPublicIp"
Select-String -Path ".\terraform.tfvars" -Pattern
"ssh_allowed_cidr"
```

The current IP must match the CIDR in `terraform.tfvars`.

Now run:

```
None
terraform apply
```

Review the plan and type `yes`.

Cost reminder
EC2 compute, its EBS disk and public IPv4 address can incur charges. We will also create a billable Load Balancer later. Don't leave the infrastructure running unnecessarily.

Step 27 — Verify EC2

After Terraform finishes successfully, run:

```
None
terraform output
```

Then:

```
None
aws ec2 describe-instances `
```

```
  --filters "Name=tag:Name,Values=terraform-microservice-ec2" `
  --region us-east-1 `
  --query
"Reservations[*].Instances[*].[InstanceId,State.Name,InstanceType,P
ublicIpAddress]" `
  --output table
```

Step 28 — EC2 deployment status

Property	Value
Instance ID	`i-0d4e75dabd33173c0`
Status	Running
Instance type	`t3.micro`
AMI	Amazon Linux 2023
Public IP	`100.48.90.254`
Availability Zone	`us-east-1a`
Key pair	`terraform-microservice-key`

Important: Running means EC2 has started. We haven't yet confirmed that Docker installed successfully or that the container is serving the page.

Step 29 — Connect to EC2 through SSH

First, check whether your public IP is still the same as the IP configured in Terraform:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()

Write-Host "Current IP: $MyPublicIp"

Select-String -Path ".\terraform.tfvars" -Pattern
"ssh_allowed_cidr"
```

The current IP must match the configured `/32` CIDR.

Now connect:

```
None
$Project = "C:\Users\HP\terraform-microservice"

ssh -i "$Project\keys\terraform-microservice-key"
ec2-user@100.48.90.254
```

At the first connection, SSH may ask whether you trust the host. Type `yes`.

Step 30 — Verify Docker and the application on EC2

Once connected, execute these commands one by one.

1. Check Docker service

```
None
sudo systemctl status docker --no-pager
```

2. Check running containers

```
None
sudo docker ps
```

3. Check downloaded images

```
None
sudo docker images
```

4. Test the application locally

```
None
curl -i http://127.0.0.1/
```

Expected HTML includes:

```
None
<h1>Hello from AWS Microservice!</h1>
```

5. Check the bootstrap log

```
None
sudo tail -n 60 /var/log/user-data.log
```

The log should contain:

```
None
Microservice deployment completed successfully
```

Your bootstrap script runs:

```
None
sudo dnf install -y docker curl unzip
```

Amazon Linux 2023 already has a minimal version of curl (`curl-minimal`). Installing the separate `curl` package creates a package conflict.

Because the script starts with:

```
None
set -euxo pipefail
```

it stops immediately when the package installation fails. That's why Docker was never installed and the container never started.

We can repair the existing EC2 instance without replacing it.

Step 31 — Correct the bootstrap script

Amazon Linux 2023 includes `curl-minimal` by default. It already provides the normal `curl` command, so we should not install the separate `curl` package.

Amazon Linux 2023

We'll remove `curl` and `unzip` from the DNF installation command. We'll also use AWS's official AWS CLI installation script as a fallback if the CLI isn't already present.

AWS Command Line Interface

31.1 Return to Windows PowerShell

At your current EC2 prompt, type:

```
None
exit
```

Then return to your Terraform folder:

```
None
Set-Location "C:\Users\HP\terraform-microservice\terraform"
```

31.2 Replace `user_data.sh`

Run this complete block:

None

```
@'
#!/bin/bash
set -Eeuo pipefail

exec > >(tee -a /var/log/user-data.log) 2>&1

echo "Starting microservice EC2 bootstrap"

# Install Docker only. curl-minimal is already available on AL2023.
dnf install -y docker

# Start Docker and enable it on boot
systemctl enable --now docker

# Install AWS CLI v2 if missing
if ! command -v aws >/dev/null 2>&1; then
    curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash -s
-- --system
fi

aws --version
docker --version

# Allow ec2-user to use Docker after reconnecting
usermod -aG docker ec2-user

# Authenticate to ECR using the EC2 IAM role
login_success=false

for attempt in {1..12}; do
    if aws ecr get-login-password --region "${aws_region}" |
       docker login --username AWS --password-stdin
"${ecr_registry}"; then
        login_success=true
        break
    fi
```

```
    echo "ECR login attempt $attempt failed. Retrying..."
    sleep 10
done

if [[ "$login_success" != "true" ]]; then
    echo "ECR authentication failed"
    exit 1
fi

# Pull application image
docker pull "${ecr_repository_url}:${image_tag}"

# Run application container
docker run -d \
    --name terraform-microservice \
    --restart unless-stopped \
    -p 80:80 \
    "${ecr_repository_url}:${image_tag}"

# Verify application
docker ps
curl -f http://127.0.0.1/

echo "Microservice deployment completed successfully"
'@ | Set-Content -Path ".\user_data.sh" -Encoding ascii
```

31.3 Check your SSH IP

Before replacing EC2, refresh the SSH allowlist:

```
None
$CurrentPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()

$tfvars = Get-Content ".\terraform.tfvars" -Raw
```

```
$tfvars = [regex]::Replace(
    $tfvars,
    '(?m)^ssh_allowed_cidr\s*=\s*".*"$',
    "ssh_allowed_cidr = `"$CurrentPublicIp/32`""
)

Set-Content ".\terraform.tfvars" -Value $tfvars -Encoding ascii

Select-String -Path ".\terraform.tfvars" -Pattern
"ssh_allowed_cidr"
```

31.4 Validate and plan

None

```
terraform fmt -recursive
terraform validate
terraform plan
```

Plan review

None

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

Resource	Action
Existing EC2 `i-0d4e75dabd33173c0`	Destroy
New EC2 instance	Create

VPC	No change
Security groups	No change
IAM role and profile	No change
ECR repository and image	No change

Important: The old EC2 instance will be terminated before the new one is created. The new instance will receive a new public IP address.

Step 32 — Replace the EC2 instance

Run:

```
None
terraform apply
```

Review the plan and enter:

```
None
yes
```

Expected final result:

```
None
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

Step 33 — Get the new EC2 details

After the apply succeeds:

```
None
terraform output
```

Get the new public IP:

```
None
terraform output -raw ec2_public_ip
```

Terraform successfully terminated the old instance and created the replacement with our corrected bootstrap script.

Step 34 — Connect to the new EC2 instance

Your new EC2 details:

Property	Value
Instance ID	`i-02d4cb4197b784fb7`
Status	Running
Instance type	`t3.micro`
Public IP	`100.60.75.61`
Availability Zone	`us-east-1a`

First, check that your current public IP still matches the SSH security-group rule:

```
None
$MyPublicIp = (Invoke-RestMethod
"https://checkip.amazonaws.com").Trim()
Write-Host "Current IP: $MyPublicIp"

Select-String -Path ".\terraform.tfvars" -Pattern
"ssh_allowed_cidr"
```

If they match, connect:

```
None
$Project = "C:\Users\HP\terraform-microservice"

ssh -i "$Project\keys\terraform-microservice-key"
ec2-user@100.60.75.61
```

Accept the new host fingerprint prompt by typing `yes`, then enter your SSH key passphrase.

Step 35 — Verify Docker installation

After connecting to EC2, run these commands one at a time.

1. Check Docker service

```
None
sudo systemctl status docker --no-pager
```

2. Check the container

```
None
sudo docker ps
```

3. Check the downloaded image

```
None
sudo docker images
```

4. Test the application

```
None
curl -i http://127.0.0.1/
```

Expected:

```
None
HTTP/1.1 200 OK
```

And the response should contain:

```
None
<h1>Hello from AWS Microservice!</h1>
```

5. Check the bootstrap log

```
None
sudo tail -n 60 /var/log/user-data.log
```

Look for:

```
None
Microservice deployment completed successfully
```

replacement EC2 instance has successfully pulled the private image from ECR and started the application.

All the important checks passed:

Check	Result
SSH connection	Successful

Docker service	Running
ECR image pull	Successful
Container	Running
Container port mapping	`80:80`
Local HTTP response	`200 OK`
Application HTML	Correct

Your running container is:

```
None
terraform-microservice
```

And the image is:

```
None
986881030460.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice-app:v1
```

One small issue in the bootstrap log: The first `curl` ran immediately after `docker run`, before Nginx was ready, and returned `Empty reply from server`. Your later manual `curl` returned `HTTP/1.1 200 OK`, so the application is now working. No EC2 replacement is necessary just for that transient readiness check.

Step 36 — Create the Application Load Balancer

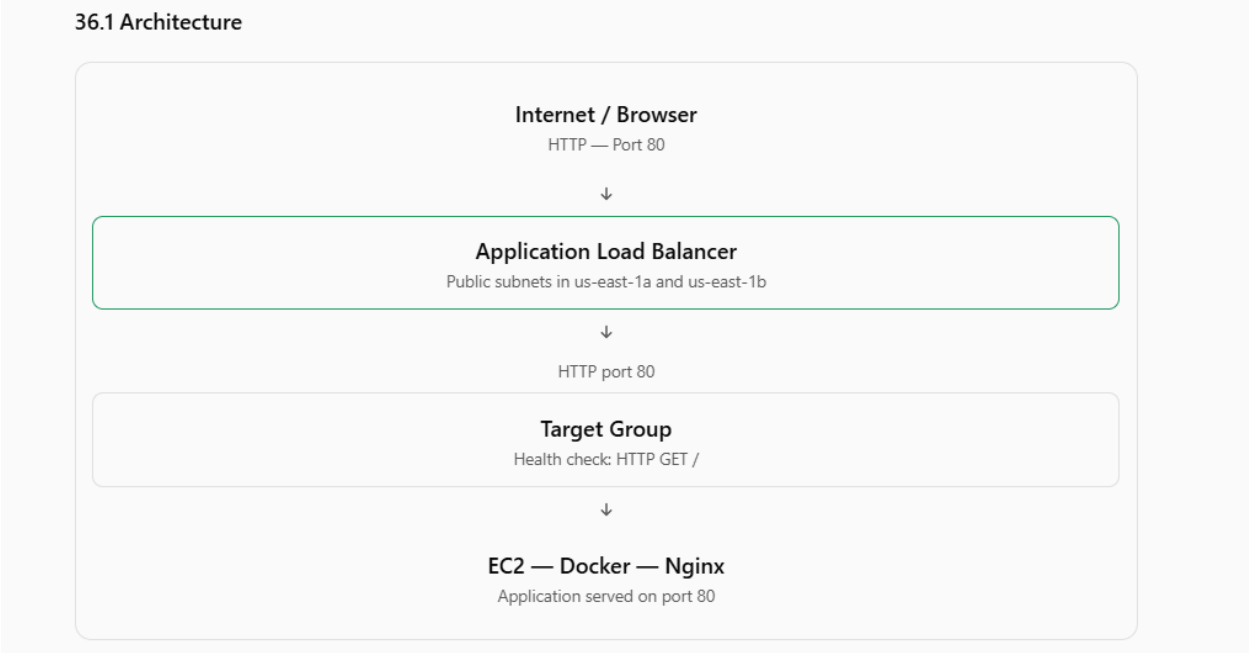
Now we'll connect internet traffic to your EC2 application.

The ALB will:

1. Receive public HTTP requests on port 80.
2. Forward requests to the EC2 instance on port 80.
3. Check whether the application is healthy.
4. Return the application's response to the user.

The target group tracks the EC2 backend, and the listener forwards incoming traffic to that target group.

GitHub+2



36.2 Create `alb.tf`

Return to your Windows PowerShell Terraform directory if you're still connected to EC2:

```
None
exit
```

Then execute this in PowerShell:

```
None
Set-Location "C:\Users\HP\terraform-microservice\terraform"
```

Create the ALB configuration:

```
None
@'
# Application Load Balancer
resource "aws_lb" "app" {
  name               = "${var.project_name}-alb"
  internal           = false
  load_balancer_type = "application"

  security_groups = [
    aws_security_group.alb.id
  ]

  subnets = [
    aws_subnet.public_a.id,
    aws_subnet.public_b.id
  ]

  enable_deletion_protection = false

  tags = {
    Name = "${var.project_name}-alb"
  }

  depends_on = [
    aws_vpc_security_group_ingress_rule.alb_http,
    aws_vpc_security_group_egress_rule.alb_to_ec2
  ]
}

# Target group for the EC2 application
resource "aws_lb_target_group" "app" {
  name        = "${var.project_name}-tg"
```

```
  port        = 80
  protocol    = "HTTP"
  target_type = "instance"
  vpc_id      = aws_vpc.main.id

  health_check {
    enabled             = true
    path                = "/"
    port                = "traffic-port"
    protocol            = "HTTP"
    matcher             = "200"
    interval            = 30
    timeout             = 5
    healthy_threshold   = 2
    unhealthy_threshold = 2
  }

  tags = {
    Name = "${var.project_name}-tg"
  }
}

# Register EC2 with the target group
resource "aws_lb_target_group_attachment" "app" {
  target_group_arn = aws_lb_target_group.app.arn
  target_id        = aws_instance.app.id
  port             = 80
}

# HTTP listener on port 80
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
```

```
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
'@ | Set-Content -Path ".\alb.tf" -Encoding ascii
```

36.3 Understand the four resources

Terraform resource	Purpose
`aws_lb.app`	Creates the public Application Load Balancer
`aws_lb_target_group.app`	Defines where application traffic goes
`aws_lb_target_group_attachment.app`	Registers your EC2 instance
`aws_lb_listener.http`	Listens on port 80 and forwards requests

The health check sends an HTTP request to `/`. If the application returns `200`, the target can become healthy.

36.4 Add ALB outputs

Append these to `outputs.tf`:

```
None
@'

output "alb_dns_name" {
  description = "Public DNS name of the Application Load Balancer"
```

```
  value       = aws_lb.app.dns_name
}

output "application_url" {
  description = "Public application URL"
  value       = "http://${aws_lb.app.dns_name}"
}

output "alb_arn" {
  description = "Application Load Balancer ARN"
  value       = aws_lb.app.arn
}

output "target_group_arn" {
  description = "Application target group ARN"
  value       = aws_lb_target_group.app.arn
}
'@ | Add-Content -Path ".\outputs.tf" -Encoding ascii
```

36.5 Validate and plan

Run:

```
None
terraform fmt -recursive
```

```
None
terraform validate
```

```
None
terraform plan
```

Step 37 — Review the four resources

Resource	Purpose
`aws_lb.app`	Creates the public Application Load Balancer
`aws_lb_target_group.app`	Defines EC2 as the application backend
`aws_lb_target_group_attachment.app`	Registers your EC2 instance
`aws_lb_listener.http`	Accepts HTTP traffic on port 80 and forwards it

Terraform confirms:

```
None
Plan: 4 to add, 0 to change, 0 to destroy.
```

Your EC2 instance ID is correctly referenced:

```
None
i-02d4cb4197b784fb7
```

The health check is configured to request `/` and expect HTTP `200`.

Step 38 — Create the Load Balancer

Run:

```
None
terraform apply
```

Review the plan and enter:

```
None
yes
```

Expected completion:

```
None
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

Cost reminder
The Application Load Balancer is billable while running. We'll verify the application and destroy the infrastructure after documenting the results.

Step 39 — Get the Load Balancer URL

After successful apply, run:

```
None
terraform output
```

Then:

```
None
terraform output -raw application_url
```

You should get an HTTP URL similar to:

```
None
http://terraform-microservice-alb-xxxxxxxx.us-east-1.elb.amazonaws.com
```

Step 40 — Check target health

Run:

```
None
aws elbv2 describe-target-health `
```

```
  --target-group-arn (terraform output -raw target_group_arn) `
  --region us-east-1 `
  --query
"TargetHealthDescriptions[*].[Target.Id,TargetHealth.State,TargetHe
alth.Reason]" `
  --output table
```

Load Balancer deployed — target healthy

This is a major milestone. Your complete application path is now configured:

```
None
Internet
   |
   v
Application Load Balancer :80
   |
   v
Target Group :80
   |
   v
EC2 Instance
   |
   v
Docker Container
   |
   v
Nginx Web Application
```

Step 41 — Verify the application through ALB

Your Load Balancer URL is:

Public application URL

HTTP 200 expected

http://terraform-microservice-alb-1542207687.us-east-1.elb.amazonaws.com

Open application

You can also test from PowerShell.

```
None
$AppUrl = terraform output -raw application_url

$response = Invoke-WebRequest -Uri $AppUrl -UseBasicParsing

$response.StatusCode
$response.Content
```

Expected:

```
None
200
```

And the HTML should contain:

```
None
<h1>Hello from AWS Microservice!</h1>
```

Step 42 — Verify direct EC2 access is blocked

This is an important security requirement of your assignment.

Run in PowerShell:

```
None
$Ec2Ip = terraform output -raw ec2_public_ip

Test-NetConnection -ComputerName $Ec2Ip -Port 80
```

Expected: blocked

Look for:

```
None
TcpTestSucceeded : False
```

This is intentional. The EC2 security group allows application traffic only from the ALB security group, not from the public internet.

Step 43 — Verify target health once more

```
None
aws elbv2 describe-target-health `
  --target-group-arn (terraform output -raw target_group_arn) `
  --region us-east-1 `
  --query
"TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]" `
  --output table
```

Expected:

```
None
i-02d4cb4197b784fb7   healthy
```

Step 44 — Final verification results

Test	Actual result	Status
ALB HTTP request	`200 OK`	Passed
Target group health	`healthy`	Passed
Application HTML	Correct page displayed	Passed

Direct EC2 port 80	`TcpTestSucceeded: False`	Passed
EC2 Docker container	Running	Passed
ECR image	`v1`	Passed

Important: Understand the direct-access test

Your result:

```
None
TcpTestSucceeded : False
PingSucceeded    : False
```

This is expected.

- `TcpTestSucceeded: False` confirms that direct HTTP access to EC2 port 80 is blocked.
- `PingSucceeded: False` means ICMP ping did not receive a response. That's not a problem.

The Load Balancer can still reach EC2 because its security group is explicitly allowed.

The `ï»¿` appearing at the beginning of the PowerShell HTML response is a harmless UTF-8 BOM. It does not affect the browser rendering or HTTP 200 result.

Step 45 — Capture project evidence

Before destroying the infrastructure, create an evidence folder on your computer.

```
None
$Project = "C:\Users\HP\terraform-microservice"
```

```
New-Item -ItemType Directory -Force -Path "$Project\evidence" |
Out-Null
```

Save Terraform outputs:

```
None
terraform output | Out-File
"$Project\evidence\terraform-outputs.txt"
```

Save the target health:

```
None
aws elbv2 describe-target-health `
  --target-group-arn (terraform output -raw target_group_arn) `
  --region us-east-1 |
  Out-File "$Project\evidence\target-health.json"
```

Save the EC2 security-group rules:

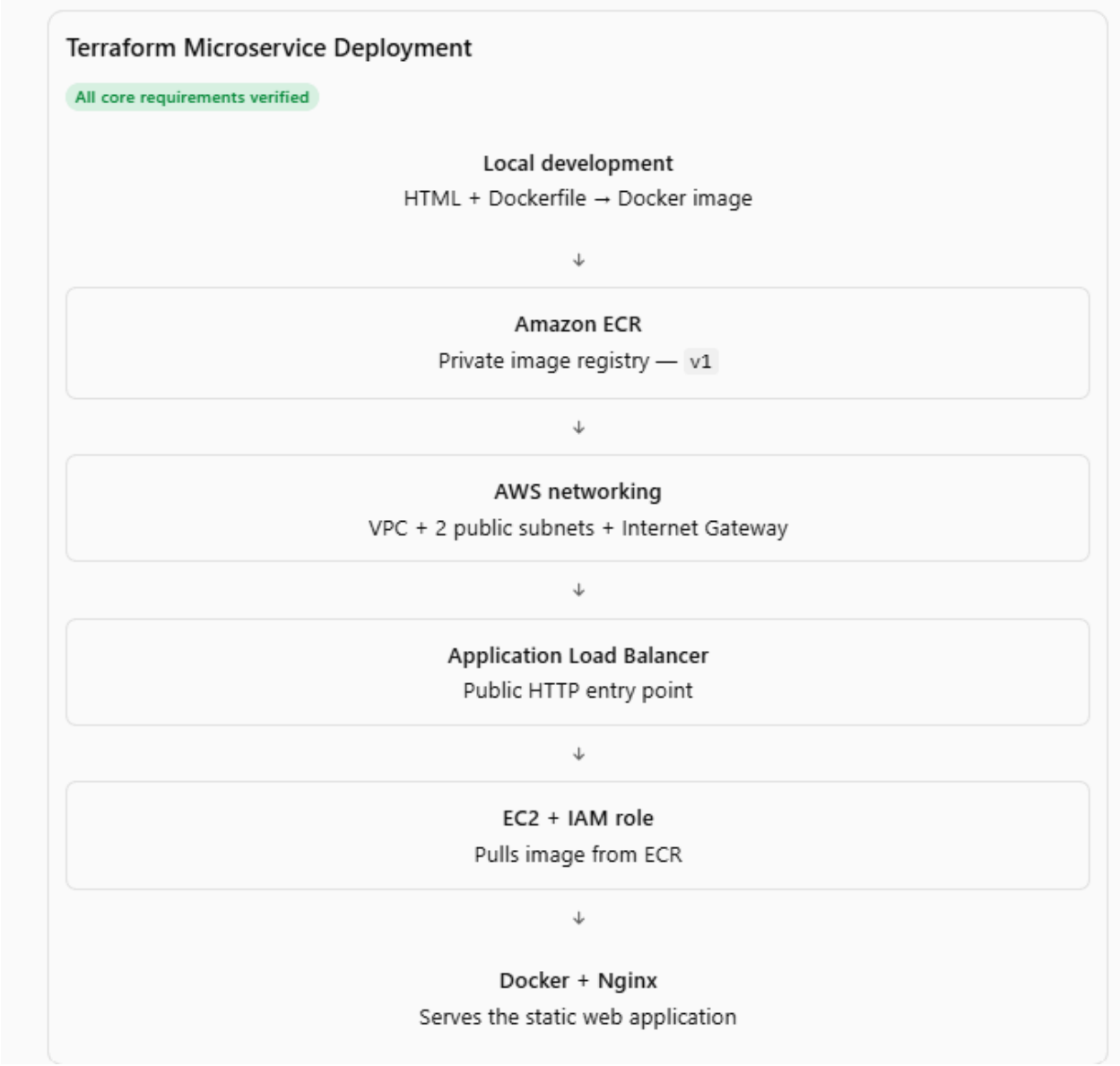
```
None
aws ec2 describe-security-groups `
  --group-ids (terraform output -raw ec2_security_group_id) `
  --region us-east-1 |
  Out-File "$Project\evidence\ec2-security-group.json"
```

Also capture these screenshots:

1. Your application page through the ALB DNS.
2. Target group showing `healthy`.
3. EC2 instance running.
4. ECR repository showing image tag `v1`.
5. Direct EC2 port 80 test showing `False`.

Step 46 — Project completion architecture

Terraform Microservice Deployment



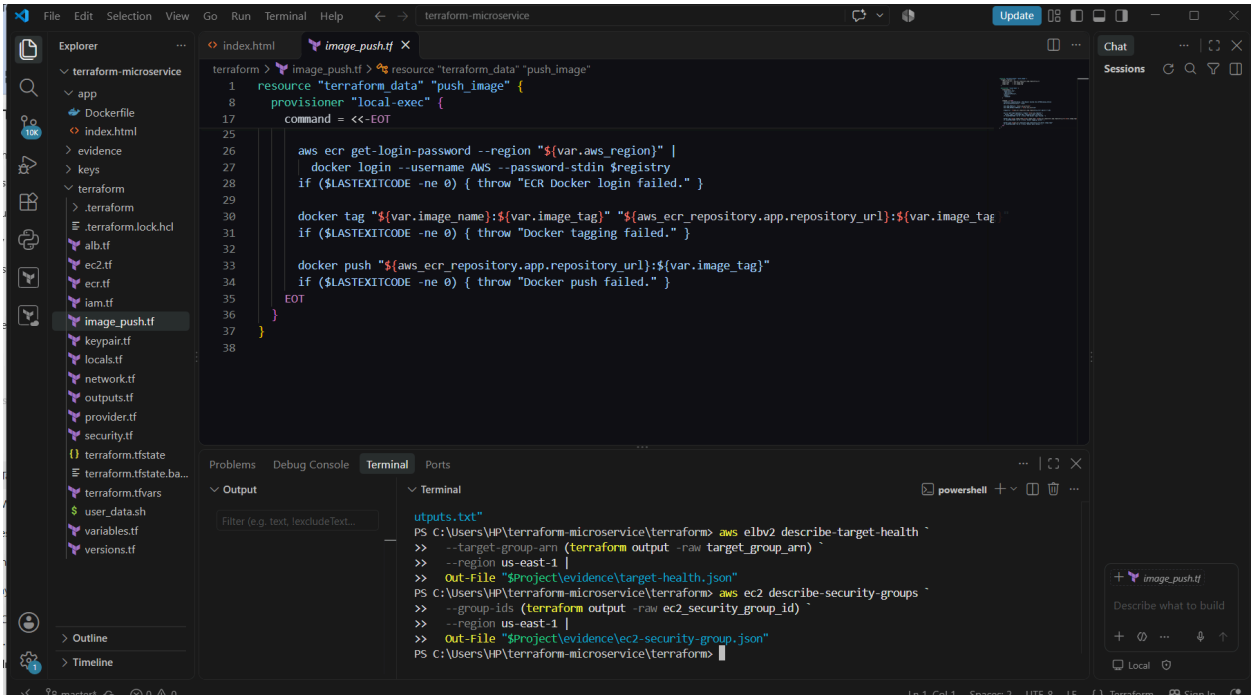
What you learned and implemented: Docker image creation, Amazon ECR, Terraform resources and dependencies, VPC networking, security groups, IAM instance profiles, EC2 user data, SSH, ALB listeners, target groups and health checks.

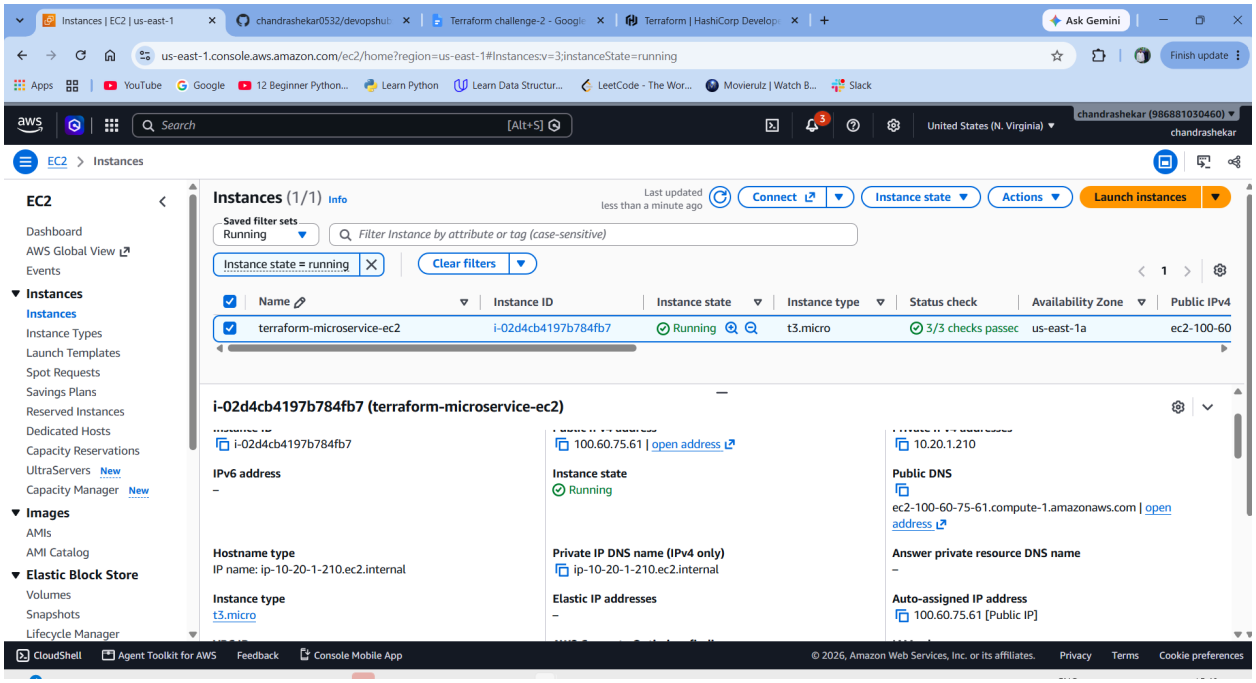
One limitation to mention in your assignment presentation: the ALB spans two Availability Zones, but there is only one EC2 backend. Therefore, this demonstrates load balancing, not full application high availability.

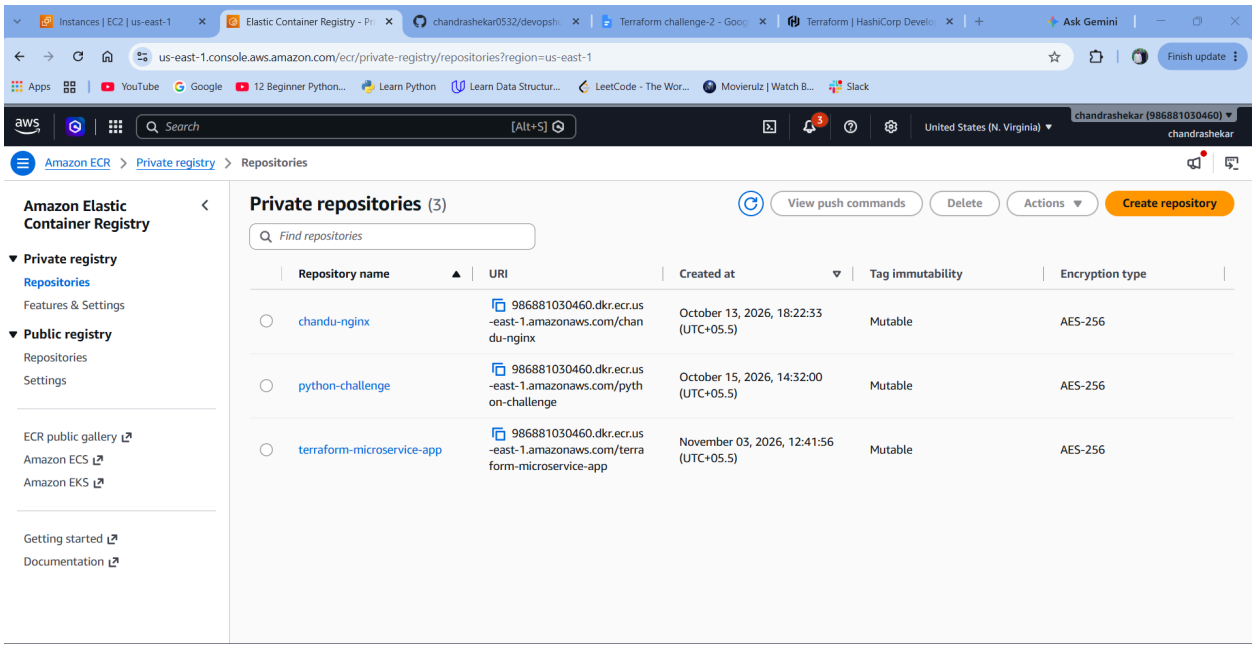
Step 47 — Clean up AWS resources

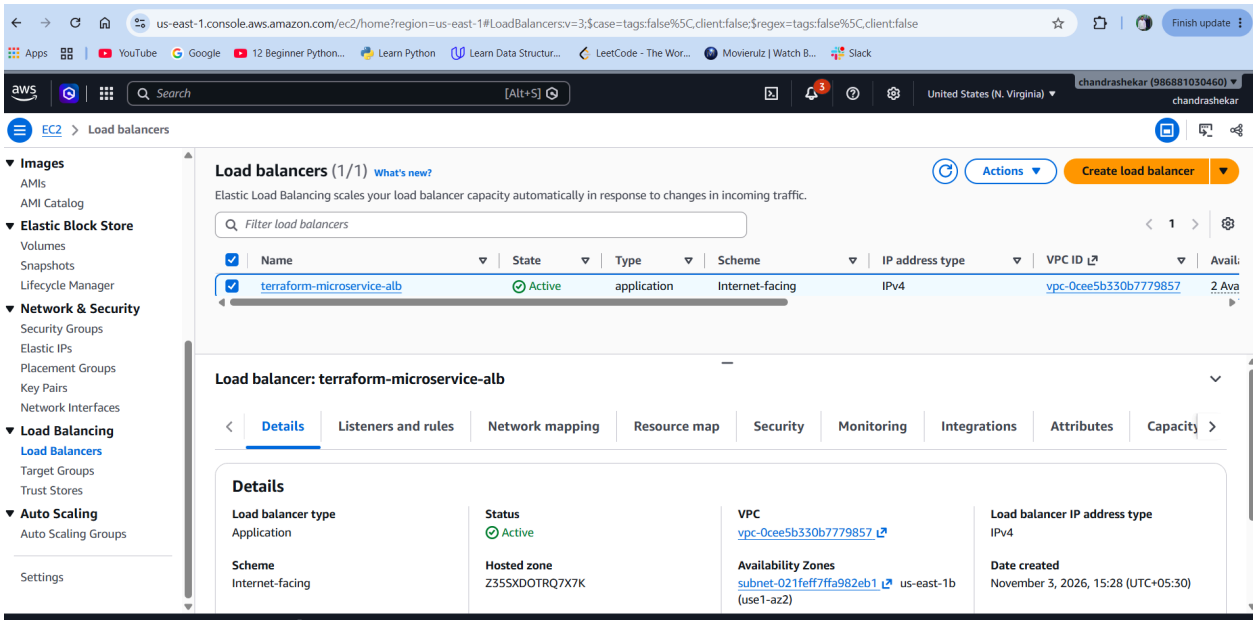
After capturing your evidence, we should destroy the training infrastructure to avoid ongoing charges.

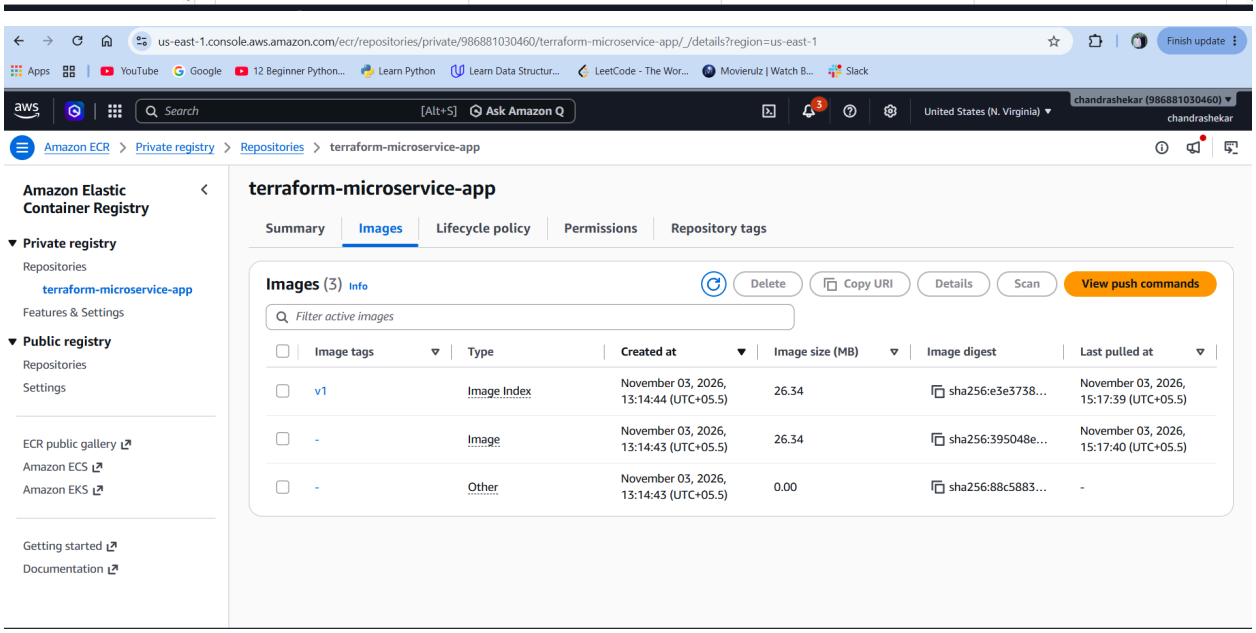
Important: `terraform destroy` will also delete the ECR repository and its uploaded image because we configured `force_delete = true`. Your local Docker image and private SSH key will remain on your computer.

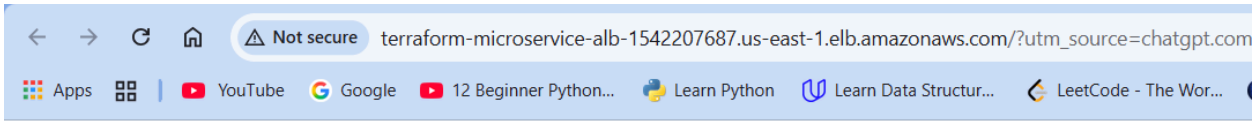



Instances | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:v=3;instanceState=running
EC2 > Instances
Instances (1/1) Info
Connect
Instance state
Actions
Launch instances
Running
Instance state = running
Clear filters
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4
terraform-microservice-ec2
i-02d4cb4197b784fb7
Running
t3.micro
3/3 checks passec
us-east-1a
ec2-100-60
i-02d4cb4197b784fb7 (terraform-microservice-ec2)
100.60.75.61 | open address
10.20.1.210
IPv6 address
Instance state
Running
Public DNS
ec2-100-60-75-61.compute-1.amazonaws.com | open address
Hostname type
IP name: ip-10-20-1-210.ec2.internal
Private IP DNS name (IPv4 only)
ip-10-20-1-210.ec2.internal
Answer private resource DNS name
Instance type
t3.micro
Elastic IP addresses
Auto-assigned IP address
100.60.75.61 [Public IP]


Elastic Container Registry - Pr
us-east-1.console.aws.amazon.com/ecr/private-registry/repositories?region=us-east-1
Amazon ECR > Private registry > Repositories
Amazon Elastic Container Registry
Private registry
Repositories
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Getting started
Documentation
Private repositories (3)
View push commands
Delete
Actions
Create repository
Find repositories
Repository name
URI
Created at
Tag immutability
Encryption type
chandu-nginx
986881030460.dkr.ecr.us-east-1.amazonaws.com/chandu-nginx
October 13, 2026, 18:22:33 (UTC+05.5)
Mutable
AES-256
python-challenge
986881030460.dkr.ecr.us-east-1.amazonaws.com/python-challenge
October 15, 2026, 14:32:00 (UTC+05.5)
Mutable
AES-256
terraform-microservice-app
986881030460.dkr.ecr.us-east-1.amazonaws.com/terraform-microservice-app
November 03, 2026, 12:41:56 (UTC+05.5)
Mutable
AES-256


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LoadBalancers:v=3;$case=tags:false%5C,client:false;$regex=tags:false%5C,client:false
Finish update
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch B...
Slack
Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
EC2 > Load balancers
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Load balancers (1/1) What's new?
Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.
Actions
Create load balancer
Filter load balancers
Name
State
Type
Scheme
IP address type
VPC ID
terraform-microservice-alb
Active
application
Internet-facing
IPv4
vpc-0cee5b330b7779857
Load balancer: terraform-microservice-alb
Details
Listeners and rules
Network mapping
Resource map
Security
Monitoring
Integrations
Attributes
Capacity
Load balancer type
Application
Status
Active
VPC
vpc-0cee5b330b7779857
Load balancer IP address type
IPv4
Scheme
Internet-facing
Hosted zone
Z35SXDOTRQ7X7K
Availability Zones
subnet-021feff7ffa982eb1 us-east-1b (use1-az2)
Date created
November 3, 2026, 15:28 (UTC+05:30)


us-east-1.console.aws.amazon.com/ecr/repositories/private/986881030460/terraform-microservice-app/_/details?region=us-east-1
Finish update
Apps
YouTube
Google
12 Beginner Python...
Learn Python
Learn Data Structur...
LeetCode - The Wor...
Movierulz | Watch B...
Slack
Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
Amazon ECR > Private registry > Repositories > terraform-microservice-app
Amazon Elastic Container Registry
Private registry
Repositories
terraform-microservice-app
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
Getting started
Documentation
terraform-microservice-app
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (3) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
v1
Image Index
November 03, 2026, 13:14:44 (UTC+05.5)
26.34
sha256:e3e3738...
November 03, 2026, 15:17:39 (UTC+05.5)
-
Image
November 03, 2026, 13:14:43 (UTC+05.5)
26.34
sha256:395048e...
November 03, 2026, 15:17:40 (UTC+05.5)
-
Other
November 03, 2026, 13:14:43 (UTC+05.5)
0.00
sha256:88c5883...
-



Hello from AWS Microservice!

This application is running inside a Docker container.

Deployed on an EC2 instance using Terraform.

Accessed through an AWS Application Load Balancer.

I have practised a complete infrastructure-as-code deployment:

Build → Containerize → Push to ECR → Provision AWS → Deploy on EC2 → Expose through ALB → Verify → Destroy.